Lopez

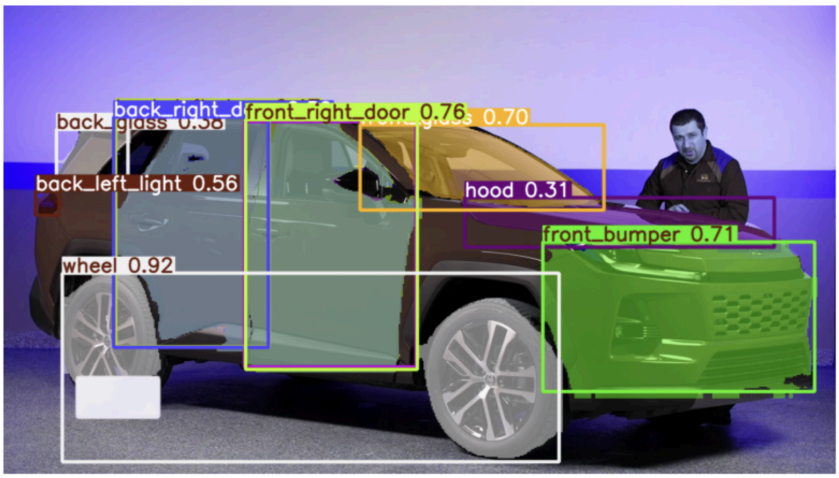
2/22/26

CS 6613 Online Assignment 2 Report

Detector Choice and Configuration

The detector I chose was the Ultralytics YOLOv8 model. I chose this model because it was designed to be fast and accurate for object detection and instance segmentation. However, the model didn't detect individual car parts like the wheel, hood, or windshield. For example, for many frames of the input video it would only detect a person and a car. Therefore, I used the Carparts segmentation dataset (made up of labeled pictures of exterior vehicle parts) to fine-tune the model to detect the classes that were missing. The model was fine-tuned for 10 epochs using transfer learning from the pretrained weights, so the training did not take very long to complete. After fine-tuning, the model was able to detect individual car parts. For each detection of a car part, the model outputs:

- a prediction for what car part it is
- a bounding box (location of car part in frame)
- a confidence score (how confident the model is in its prediction)



Fine-tuned YOLOv8 detector output from a frame in the input video

Video Sampling Strategy

To generate the frames for the Parquet file with the video detections, I extracted one frame from the input video every 20 seconds. Initially, I tried sampling every 5 seconds; however, the processing time took much longer than I expected and many of the frames were very similar because the scene did not change much between them. The video is fairly long and the presenter often talks a lot in between scene changes. With 20 second sampling, I ended up with 140 frames in total. I also cut down on the processing time by downscaling the frames to 1280x720 before running the detector. This resolution proved to have enough detail to produce good detection results.

Each frame represents the video at a specific moment in time and is associated with a frame number and timestamp. After extracting the frames from the input video, the detector was run on each one to identify car parts. Each car part detection was then associated with the frame and timestamp from which it came and stored in a searchable index suitable for retrieval later on. This index is stored in the video_detections.parquet file.

Image-to-Video Matching Logic

To process a query image, the detector can be run on the image to detect car parts. For each detected part in the query image, the video_detections parquet file can be searched to retrieve the number of detections (a sum) in the input video for that specific car part, along with the start and end timestamps of continuous intervals where the part appears in the input video. Those intervals can be computed from timestamps of sampled frames in a row where the part was detected.

The video_id column supports scalability, since the index could theoretically contain detections from multiple videos. This would allow a user to search across different videos and identify which specific video a detection is coming from. I tested the index by querying some images from the provided rav4-exterior-images dataset and verified the results by checking the output timestamps in the input video to see whether the detected car part in the query image was present. Overall, the results were mostly accurate.

Failure Cases and Limitations

The most obvious limitation is the sampling rate of the input video. Since frames were extracted at fixed intervals, the query image could return a time range that does not always show the car part, because it may appear or disappear between sampled frames. In my case, 20 seconds could be considered long; however, for the input video provided it may not be as problematic since as previously mentioned, the video takes its time between scene changes. A faster sampling rate would likely lead to better results, however, it would also increase the processing time.

Another limitation is the detector itself, since it is not perfect and can potentially produce false detections or just flat out miss some detections within frames. The index is heavily dependent on the detector on both ends: both when building the index and when querying it. The fine-tuning I did was fairly straightforward, and I did not experiment much with different parameters, so the model could likely be improved.